

Problem Solving Practice

HW Problem #3

CSE4152
Sogang University



Matrix Addition

There is an $N \times N$ matrix A initialized with arbitrary values. You are given Q queries. Each query consists of five values $R1, R2, C1, C2, V$, where $1 \leq R1, R2, C1, C2 \leq N$ and V is a real number. Consider the following program.

```
for  $k := 1$  to  $Q$  do  
    read ( $R1, R2, C1, C2, V$ ) ;  
    for  $i := R1$  to  $R2$  do  
        for  $j := C1$  to  $C2$  do  
             $A[i][j] := A[i][j] + V$  ;  
        end  
    end  
end
```

If this program is executed, it takes $O(QN^2)$ time. Design a more efficient method.

Matrix Addition

You are given an $N \times N$ matrix A initialized with arbitrary values. Several operations will be provided, each defined by five values: $R1$, $R2$, $C1$, $C2$, and V . For each operation, you need to update the matrix by adding V to all elements in the submatrix defined by the rows from $R1$ to $R2$ and the columns from $C1$ to $C2$.

Input Format:

- An integer N representing the size of the matrix ($N \times N$).
- N lines containing the matrix A , with each line containing N values.
- An integer Q representing the number of operations.
- For each of the Q operations, five values are provided: $R1$, $R2$, $C1$, $C2$, and V .
 - $R1 \leq R2$ represents the range of rows.
 - $C1 \leq C2$ represents the range of columns.
 - V is the value to be added to the specified submatrix.

Output Format:

- After all the operations, print the updated matrix A . Each matrix value should be printed with 12 decimal places of precision.

Rainbow Path

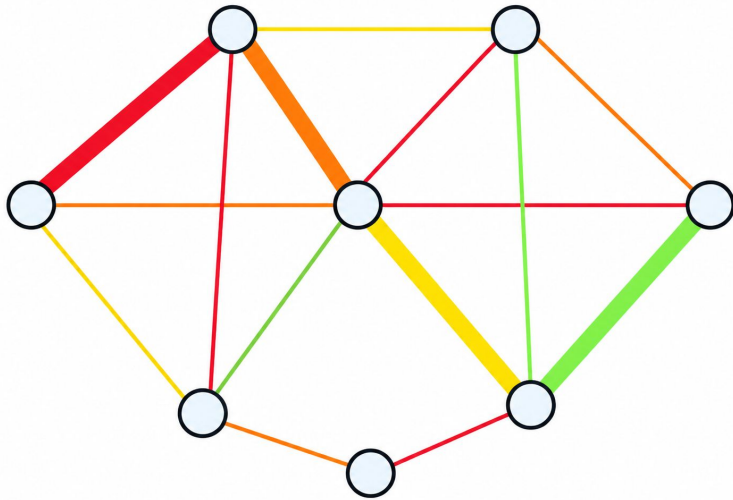
Each edge in the graph has a weight w_e and a color c_e .

Among all Rainbow Paths from S to T , find the one with the minimum total weight.

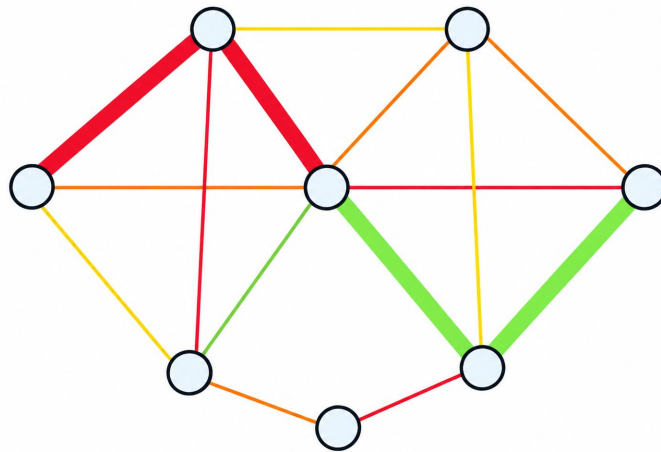
A Rainbow Path is a path whose edge colors are non-decreasing along the direction of travel.

That is, if the path consists of $e_1, e_2, \dots, e_L$, then it must satisfy $c_{e_1} \leq c_{e_2} \leq \dots \leq c_{e_L}$.

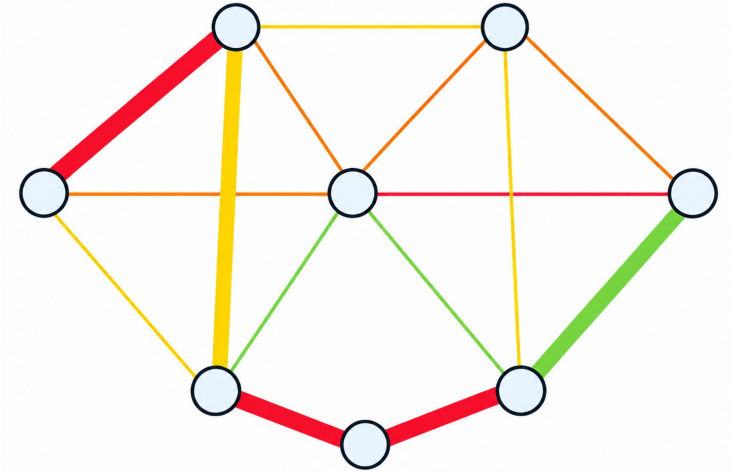
N , M , and K denote the number of vertices, edges, and colors, respectively.



rainbow path



rainbow path



NOT rainbow path

Rainbow Path (Guide Questions)

1. First, propose a simple algorithm for this problem, even if it is too slow for the given constraints. Verify that it always finds the correct shortest Rainbow Path, and analyze its time complexity.

2. Suppose that when reaching a vertex v , we also store the color c of the last edge used.

If we expand the graph using this state and compute the shortest path, what is the time complexity?

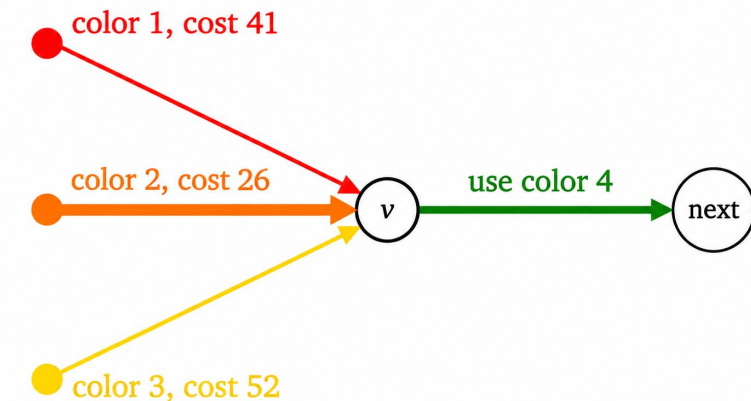
3. Suppose the last edge used has color c .

Do we still need to consider edges with colors $1, 2, \dots, c-1$ afterward?

4. Suppose we are currently at vertex v , just before using an edge of color 4.

The following costs for reaching v are known:

- Cost 41 using color 1
- Cost 26 using color 2
- Cost 52 using color 3



Do we need to keep all three values? If not, which value should be kept?

5. Based on the observations above, propose an algorithm that improves upon the algorithm from Question 1, 2, and analyze its time complexity.

Confusing change

There are N customers. After making a purchase, each customer must receive an amount of change between 0 and $M(M \leq 3)$, inclusive.

You must prepare a set of coins in advance to distribute among the customers. Each coin may have any positive integer value, and you do not know beforehand how much change each customer will require.

Determine the minimum number of coins that must be prepared so that every possible combination of change amounts can always be handled.

Challenge Question: Solve the problem for general M .
(you do not need to solve it)

Example 1

$N = 2, M = 2$

Possible requests:

(1, 1), (1, 2), (2, 1), (2, 2)

Minimum number of coins: **3**

Examples of optimal sets: (1, 1, 2)

Example 2

$N = 2, M = 4$

Possible requests:

(1, 1), (1, 2), (1, 3), (1, 4), (4, 3), (4, 4) 16 possible requests

Minimum number of coins: 5

Examples of optimal sets: (1, 1, 2, 2, 2) or (1, 1, 2, 2, 3) or ...

Confusing change (Guide Questions)

1. Consider the cases where the total amount of change required is exactly 1, then 2, and so on up to N .
How can the coins be prepared so that every possible distribution for each total amount can be handled?
2. Next, consider the cases where the total amount of change required is between $N+1$ and $2N$.
How should the coin set be extended so that every possible distribution in this range can be handled?
3. Now consider the cases where the total amount of change required is between $2N+1$ and $3N$.
How should the coin set be extended to handle every possible distribution in this range?
4. Based on the observations above, design an algorithm for the case $M \leq 3$, and analyze its time complexity.

Challenge Question: Solve the problem for general **M**.
(you do not need to solve it)

Confusing change (Guide Questions)

The guide problems for the challenge, so you do not need to solve them.

1. Consider the case where there is only one customer $N = 1$. How does the minimum number of coins required depend on M ?
2. Now suppose that each customer requires at most 1 unit of change $M = 1$.
When $M = 1$, what is the minimum number of coins required, and what coin values should be prepared?
3. Now consider the case $M=2$. Are the coins prepared for $M=1$ still necessarily required when $M = 2$?
If so, what additional coins should be added while keeping those coins?

For the following questions, assume that coins are chosen in increasing order of value.

4. Specifically, let $N = 4$, and suppose that four 1-unit coins have already been prepared. If the value of the next coin must be chosen from 1, 2, 3, which value is the best choice when $M = 2$, and which value is the best choice when $M = 3$? Explain why.
5. Now let $N = 4$, and suppose that the current set of coins is (1, 1, 1, 1, 2, 2). If the value of the next coin must be chosen from (2, 3, 4), which value is the best choice when $M=3$? What changes when $M \geq 4$? Explain why.
6. Let S be the total value of the coins prepared so far. Suppose that the current set of coins can handle every possible request from the N customers whose total amount of change is at most S . Let X be the value of the next coin to be added. By considering the case where all customers require the same amount of change, determine the maximum possible value of X .
7. As an induction hypothesis, assume that the current set of coins can handle every case in which the total amount of change required is at most S . After adding one coin of value X , show that every case in which the total amount of change required is at most $S+X$ can also be handled. Consider the following two cases:
 - a. Some customer requires at least X . -> Show that this case can be handled by using the newly added coin.
 - b. Every customer requires less than X . -> Show that this case can be handled without using the newly added coin.
8. Based on the above observations, design an algorithm and analyze its time complexity.